Hospital Queue & Bed Management System — 10-Day Build Plan (Production-Shaped & AP-Aligned)

Team of 3 · Phase 1 Foundation (Days 1–2): Sat 24 Oct – Mon 2 Nov 2026 [COMPLETED] · Active Execution (Days 3–10): Tue 3 Nov – Tue 10 Nov 2026

---

1. Roles & Domain Ownership

Each person owns one operational domain end-to-end (database, API, UI, and automated tests) **and receives an equal, balanced share of all 9 technology layers**: HTML, CSS, React, JS, Node.js, Raw SQL, Prisma ORM, Python, and Automated Testing.

Person	Teammate	Operational Domain	End-to-End Ownership Scope
Person A	**Ishika**	**Beds & Admissions**	Wards, physical beds, bed assignments, bed status lifecycle, concurrency locking, admission/transfer/discharge workflows.
Person B	**Aarpan**	**OPD Queue & Wait-time Analytics**	OPD queues, daily patient tokens, priority triage, doctor sessions, consultation logging, Erlang-C wait-time modeling.
Person C	**Chinmaya**	**Platform, Patients, Auth & City-wide View**	Express skeleton, middleware pipeline, Argon2id/JWT auth, patient identity & UHID, audit logging, idempotency, city capacity view.

---

2. Split by Technology (Equal-Share Matrix)

Every teammate works actively across all 9 technical disciplines. Work already completed across branches is marked as `[COMPLETED]`.

Technology Layer	Person A: Ishika (Beds & Admissions)	Person B: Aarpan (OPD Queue & Analytics)	Person C: Chinmaya (Platform, Patients & Auth)
SQL & Migrations	• `[COMPLETED]` `002_ward_bed_assignment.sql` (commit `d0e0e21` on `feat/day1-bed-management-clean`) • Concurrency lock: `SELECT ... FOR UPDATE SKIP LOCKED` • Partial unique index on active bed & active patient • Bed status transition trigger • `withTransaction()` transaction helper	• `003_queue_core.sql` (migrations for `opd_queue`, `queue_entry`, `queue_counter`, `doctor_session`, `consultation_log`) • Atomic token sequence generator via `queue_counter` • Compare-and-swap transition (`WHERE status = 'WAITING'`) • Priority aging calculation query • PostgreSQL views: arrival rate, service time, live doctor count	• `[COMPLETED]` `001_identity_core.sql` (commit `0b01c85` on `person-c`) • `004_roles_and_rls.sql` • Non-owner DB role (`api_user`) • City aggregate role (`city_reader`) • Row Level Security (RLS) policies on `patient`, `admission`, `queue_entry` • `v_hospital_capacity` aggregate view

Technology Layer	Person A: Ishika (Beds & Admissions)	Person B: Aarpan (OPD Queue & Analytics)	Person C: Chinmaya (Platform, Patients & Auth)
Prisma ORM	• Models: `Ward`, `Bed`, `BedAssignment` • Relations: `Hospital` $\to$ `Ward` $\to$ `Bed` $\to$ `BedAssignment` • Prisma Client queries: `bed.findMany` with status filtering & ward sorting • Seed generator: Ward & Bed initial seed data	• Models: `OpdQueue`, `QueueEntry`, `DoctorSession`, `ConsultationLog` • Relations: `Department` $\to$ `OpdQueue` $\to$ `QueueEntry` • Prisma Client queries: `queueEntry.findMany` with pagination (`skip` / `take`) & priority sorting • Seed generator: Queue & Doctor initial seed data	• Models: `City`, `Hospital`, `Department`, `AppUser`, `UserHospitalRole`, `Patient`, `AuditLog`, `IdempotencyKey` • Relations: `User` $\to$ `Roles`, `Patient` $\to$ `Hospital` • Prisma Client singleton (`src/lib/prisma.js`) • Duplicate detection query (`findFirst` by name + DOB) with pagination • Master seed runner (`prisma/seed.js`)
Node / Express (MVC Layer)	• Controller: `bed.controller.js` • Service: `bed.service.js` • Route: `bed.routes.js` • Bed reservation expiry sweeper • Housekeeping status transition handlers	• Controller: `queue.controller.js` • Service: `queue.service.js` • Route: `queue.routes.js` • Doctor session start/pause/end admin • "Symptoms worse" triage escalation handler	• **`[COMPLETED]`** `api/package.json` setup (Express 5, pg, nodemon) • Express app assembly (`app.js`) & server listener (`server.js`) • Controllers & Services: `auth`, `patient`, `city` • Routes: `auth.routes.js`, `patient.routes.js`, `city.routes.js` • API versioning (`/api/v1/...`)
Middlewares & Validation	• Zod schemas for bed booking, transfer & discharge (`bedValidation.js`) • Bed state-machine transition guard middleware	• Zod schemas for queue join, token call & priority triage (`queueValidation.js`) • Triage priority boundary guard middleware	• 9-stage middleware pipeline assembly (`app.js`) • `helmet()`, `cors()`, `express.json()`, `rateLimit()` • `pino-http` logger with UUID request-id & PHI sanitization • JWT auth guard & RBAC hospital-scoping middleware • Idempotency middleware (`idempotency_key` table) • Centralized error handler `(err, req, res, next)`
Node Core Runtime (Events & Streams)	• Emits `bedReleased` event upon patient discharge and housekeeping clearance • Listens to `emergencyAdmit` event to auto-hold ICU beds	• Listens to `bedReleased` event to auto-dispatch waitlisted patients • Emits `patientCalled` and `symptomsEscalated` events • Background interval timers for queue rolling averages	• Central `EventEmitter` bus singleton (`src/lib/events.js`) • Node.js `Readable` streams for memory-efficient audit-log export • Node `process` signal handlers (`SIGTERM` / `SIGINT`) for graceful shutdown

Technology Layer	Person A: Ishika (Beds & Admissions)	Person B: Aarpan (OPD Queue & Analytics)	Person C: Chinmaya (Platform, Patients & Auth)
Python / FastAPI (Analytics)	• `analytics/bed_analytics.py`: Bed turnover & ALOS (Average Length of Stay) prediction engine • Ward depletion forecast & ICU surge risk model • FastAPI router: `POST /analytics/beds/forecast` & `POST /analytics/beds/turnover` • Pytest test suite: `tests/test_bed_analytics.py` (ALOS, zero-capacity, high discharge edge cases)	• `analytics/queue_engine.py`: Erlang-C multi-server ($M/M/c$) queue math formulas • Dynamic wait-time range calculator & doctor saturation index ($\rho = \lambda / (c \cdot \mu)$) • FastAPI router: `POST /analytics/queue/estimate` & `POST /analytics/queue/saturation` • Pytest test suite: `tests/test_queue_engine.py` (Erlang-C edge cases, $\rho \ge 1$, $c=0$, Erlang-C table verification)	• `analytics/city_analytics.py`: Regional hospital strain index & anomaly detector • Cross-hospital load-balancing optimizer & referral recommendation engine • FastAPI microservice orchestration (`main.py`, auth middleware, shared secret validation, `/health`) & router: `POST /analytics/city/strain` • Pytest test suite: `tests/test_city_analytics.py` (strain scoring, multi-hospital distance-capacity matrix, outlier handling)
React Frontend	• Ward bed map with live occupancy badges • Admit / discharge / transfer modal dialogs • Housekeeping sanitation board • Bed reservation countdown timer	• Staff queue board with calling actions • Doctor calling console • Patient token screen with "symptoms worse" button • Live queue stats widget	• App shell with `react-router-dom` & protected route guards • Login screen with lockout feedback • Patient registration UI with duplicate warning modal • City capacity dashboard & referral UI • Audit-log viewer
HTML (Semantic Markup & JSX)	• Semantic Ward grid & isolation cubicles layout markup • Bed card component structure (`BedCard.jsx`) • Inpatient modal forms (Admit, Transfer, Discharge) • Housekeeping turnover checklist markup	• Doctor & staff calling console table markup • Public waiting-room kiosk layout (large-screen display, tokens only, zero PHI) • Digital token slip with QR structure & emergency triage trigger markup	• Global app shell structure (`<header>`, `<nav>`, responsive sidebar drawer, breadcrumbs) • Accessible auth markup (Login form, lockout banner, role selector) • Master patient registration & UHID search forms • City-wide multi-hospital capacity overview cards
CSS (Design Tokens & Stylesheets)	• `beds.css`: Ward CSS grid layout (mobile to desktop), bed occupancy status badges, modal slide-up keyframe transitions • Isolation room warning patterns & sanitation clearance animations	• `queue.css`: High-contrast public TV display theme (WCAG AAA), active token pulse animation, calling sound visualizer wave, triage priority indicators	• `tokens.css`: Design tokens, color system, typography scale • `layout.css` & `forms.css`: App shell layout, accessible form inputs, validation error banners, login shake animation
Automated Testing	• Concurrency test script (20 simultaneous requests against 5 beds in CI) • Pytest bed forecasting & turnover analytics test suite	• k6 load test script on queue join & call-next • Queue FIFO order & priority aging unit tests • Pytest Erlang-C queue mathematical test suite	• **`[COMPLETED]`** GitHub Actions CI pipeline (`.github/workflows/ci.yml`) • Supertest suite for authentication & lockout • Supertest suite for tenant isolation (cross-hospital 403) • Pytest city surge & referral optimizer test suite

Shared Reusable Pieces

- **Person A owns**: `Badge`, `Table`, `Modal` accessible UI components (`[COMPLETED]` on `feat/day1-bed-management-clean`).
- **Person B owns**: Shared API client, `usePolling` hook with stale-data indicator, date/wait-time formatters.
- **Person C owns**: Express app skeleton, middleware pipeline, database pool/Prisma singletons, auth & audit helpers.

---

3. Production-Readiness Requirements

These turn the academic prototype into a production-grade hospital system:

Requirement	What Gets Built	Owner	Day	Status
Identity & Access	`app_user`, per-hospital roles, Argon2id hashing, 5-attempt lockout, login rate limiting, JWT tokens, route guards.	C	2	Schema done; Auth routes pending
Real Authorization	Every query scoped by hospital; API runs as non-owner `api_user` role with RLS; city dashboard reads aggregate view through `city_reader`.	C	3–4	Pending
Audit Trail	Append-only `audit_log` written on sensitive PHI views/exports, admit, transfer, discharge, queue call, and referral; DML revoked; admin viewer.	C (helper), A & B (call it)	2–7	Schema done; Helper pending
Patient Identity	UUID IDs, unique UHID per hospital, date of birth, duplicate check warning, merge pointer.	C	1–2	Schema done; Routes pending
Bed Integrity	`bed_assignment` history, composite FKs, status transition triggers, reservation expiry sweeper.	A	1–4	Migration done; Sweeper pending
Safe Retries & Races	`Idempotency-Key` on admit, join, and referral; atomic token counter; compare-and-swap status updates; row locks.	C (idempotency), A (beds), B (queue)	3	Schema done; Middleware pending
Emergency & Fairness	Defined priority levels, priority aging formula so low priority is never starved, audited emergency flag, "symptoms worse" escalation.	B	3–7	Pending
Honest Wait Estimates	`doctor_session` gives live c; FastAPI returns a range behind a shared secret; Node falls back if Python is down.	B (service), C (client)	4	Pending
Privacy	Waiting-room display shows tokens only; no PHI in logs or URLs; masked phone numbers in lists; consent notice at registration.	B, C	6–8	Pino redaction planned
Migrations & CI	All SQL migrations through `node-pg-migrate` (single migration owner) with Prisma model synchronization (`prisma db pull`); GitHub Actions builds fresh Postgres 16 and runs every test on each PR.	C (setup), all	1	CI ready; 001/002 done; 003–004 pending
Automated Tests	Concurrency & bed tests (A), queue & Erlang-C tests (B), authorization & end-to-end tests (C).	A, B, C	3–9	CI ready; Tests pending

Requirement	What Gets Built	Owner	Day	Status
Ops Basics	`/health` endpoint, separate dev and prod config, backup-and-restore drill, written paper-fallback downtime procedure.	C (drill), A (procedure)	8–9	Config done; Drill pending

Documented in the report, not built: Full DPDP consent management program, MFA, high availability clustering, ABDM/ABHA integration, appointment scheduling, time-varying wait-time models, clinical diagnostic validation.

---

4. Day-by-Day Timeline

Phase 1 — Foundation (Days 1–2)

- **Milestone**: End of Day 1: Migrations apply cleanly on fresh Postgres in CI and all foundations merged to `main`. End of Day 2: Login works, seed data populates, and staff users only see their own hospital's rows.

Day	Person A: Ishika (Beds)	Person B: Aarpan (Queue)	Person C: Chinmaya (Platform & Auth)
Day 1 Sat 24 Oct [COMPLETED]	**`[COMPLETED]`** `002_ward_bed_assignment.sql` **`[COMPLETED]`** `tokens.css` **`[COMPLETED]`** `Badge.jsx`, `Modal.jsx`, `Table.jsx` UI components	**`[COMPLETED]`** Repo structure alignment • Write `003_queue_core.sql` • API client + `usePolling` hook • FastAPI skeleton (`/health`, shared secret)	**`[COMPLETED]`** Repo init & `api/package.json` **`[COMPLETED]`** `001_identity_core.sql` **`[COMPLETED]`** CI workflow (`ci.yml`) **`[COMPLETED]`** Proposal overhaul • Express skeleton (`app.js`, `server.js`) • Merge `feat/day1` branch & setup Vite shell
Day 2 Sun 25 Oct – Mon 2 Nov [COMPLETED]	• Bed Prisma model & relations • Bed routes (`GET /beds`, status updates) • Bed state-machine helper • Bed seed generator	• Queue Prisma model & relations • `POST /queue/join` with atomic counter • `GET /queue/:id` • Priority levels defined in `API.md`	• Auth routes: register & login (Argon2id) • Account lockout (5 attempts) & JWT signing • RBAC & hospital-scoping middleware • Patient routes with UHID & duplicate check • Master seed runner (`prisma/seed.js`)

Phase 2 — Core Backend (Days 3–5 morning)

- **Milestone**: End of Day 5 Morning: Backend checkpoint. CI is green, cross-authorization penetration tests pass, and API contract `backend-v1` is frozen.

Day	Person A: Ishika (Beds)	Person B: Aarpan (Queue)	Person C: Chinmaya (Platform & Auth)
Day 3 Tue 3 Nov	• `admitPatient`, `discharge`, `transfer` routes with `FOR UPDATE SKIP LOCKED` • `withTransaction()` integration • Concurrency test (20 requests vs 5 beds) in CI	• `call-next` route with priority aging • Compare-and-swap transitions (`WHERE status = 'WAITING'`) • No-show / hold logic • Audited emergency flag & queue unit tests	• Idempotency middleware on admit, join, referral • Migration `004_roles_and_rls.sql` • Switch API to non-owner `api_user` role • Enable RLS on `patient`, `admission`, `queue_entry` • Cross-hospital authorization tests
Day 4 Wed 4 Nov	• Waitlist table & auto-assign logic • Housekeeping routes (`CLEANING` $\to$ `AVAILABLE`) • Reservation-expiry sweeper • Bed analytics engine (`analytics/bed_analytics.py`) & Pytest suite	• Erlang-C mathematical formula in FastAPI (`analytics/queue_engine.py`) • `POST /estimate` returning wait range • Live `doctor_session` counter • Queue analytics Pytest suite (`tests/test_queue_engine.py`)	• `city_reader` role + aggregate view `v_hospital_capacity` • Referral routes with expiry • City strain & load-balancing engine (`analytics/city_analytics.py`) & Pytest suite • Node $\to$ FastAPI client with timeout & fallback
Day 5 AM Thu 5 Nov (Morning)	**Backend Checkpoint (All Together)**: Try to break teammate's authorization: cross-hospital reads, unauthenticated mutations, replayed idempotency requests. Fix findings, tag `backend-v1`.		

Phase 3 — Frontend (Days 5 afternoon–7)

- **Milestone**: End of Day 7: Full patient journey operable in the browser: intake $\to$ queue $\to$ consult $\to$ bed allocation $\to$ discharge $\to$ housekeeping.

Day	Person A: Ishika (Beds)	Person B: Aarpan (Queue)	Person C: Chinmaya (Platform & Auth)
Day 5 PM Thu 5 Nov (Afternoon)	• Ward bed map screen + CSS • Live occupancy indicators (color + text)	• Staff queue board + CSS • Calling controls	• React App shell, router, route guards • Login screen with lockout feedback
Day 6 Fri 6 Nov	• Admit, discharge, and transfer modals • Bed reservation countdown timer	• Doctor calling console • Patient token screen • Waiting-room display (tokens only, no PHI) • "Symptoms worse" button	• Patient registration form with duplicate modal • Patient consent notice page • User and role admin screens
Day 7 Sat 7 Nov	• Housekeeping sanitation board • Bed waitlist view • Mobile-responsive bed layout	• Queue stats widget • Priority aging indicator • Stale-data indicator	• City capacity dashboard (aggregates with timestamp) • Inter-hospital referral UI • Audit-log viewer screen

---

Phase 4 — Hardening & Freeze (Day 8)

- **Milestone**: End of Day 8: Feature freeze. Full end-to-end rehearsal from a fresh database under different user roles.

Day	Person A: Ishika (Beds)	Person B: Aarpan (Queue)	Person C: Chinmaya (Platform & Auth)
Day 8 Sun 8 Nov	• Bed & ward admin screen • Transfer edge cases (double discharge, zero beds free) • Concurrency bug fixes	• k6 load test on queue join & call-next • Wait-time model calibration against sample clinic data • Department & doctor session admin	• Security pass: helmet headers, CORS allow-list, rate limits, no PHI in logs • Playwright end-to-end automated test • Backup $\to$ restore drill on clean database

---

Phase 5 — Testing, Documentation & Demo (Days 9–10)

Day	Person A: Ishika (Beds)	Person B: Aarpan (Queue)	Person C: Chinmaya (Platform & Auth)
Day 9 Mon 9 Nov	• Final concurrency edge-case tests • ER diagram v2 + DFD 2B • Written paper-fallback downtime procedure • Bed integrity section of project report	• Load-test results & token-uniqueness proof • Wait-time model limits & assumptions • DFD 2A (Queue workflow) • Analytics section of project report	• README (setup, env, migrations, roles) • DFD 0, 1, 2C, 2D • Threat model & privacy documentation • Report assembly & demo seed dataset
Day 10 Tue 10 Nov	• Buffer & bug fixes • Rehearse bed & admission live demo	• Buffer & bug fixes • Rehearse queue & analytics live demo	• Buffer & bug fixes • Presentation slides • Rehearse platform, auth & city demo

---

5. Scope Tiers (Cut from the Bottom if Behind)

- **Must Have**: Migration-built schema, Prisma models, users/roles/auth with hospital scoping, Argon2id hashing, lockout, audit log, bed reservation with locking (`SELECT ... FOR UPDATE SKIP LOCKED`), queue join and call-next with atomic tokens, Erlang-C range estimate, bed map, queue board, token screen, automated tests in CI.
- **Should Have**: Waitlist auto-assignment, city dashboard on aggregate role, referral workflow, admin screens, audit-log viewer, duplicate-patient warning, k6 load test.
- **Could Have**: Supabase realtime (use 5-second polling instead), patient merge UI, deployment to Render/Vercel.
- **Documented, Not Built**: Full DPDP consent management program, MFA, high availability clustering, ABDM/ABHA integration, appointment scheduling, time-varying wait-time models, clinical diagnostic validation.

---

6. Working Rules

1. **Day 1 cannot slip**: Foundations (`001`, `002`, Express skeleton, CI) must be merged to `main` before Day 2 feature work starts.
2. **Schema changes only through migrations**: No hand-edited tables in the Supabase UI editor. Every change goes through versioned migration files and Prisma schema updates.

3. **Merge to `main` at least once a day**: Every PR requires review by one teammate (A reviews B, B reviews C, C reviews A) and a passing CI run.
4. **10-minute daily standup**: 1) What I finished, 2) What I am working on today, 3) What is blocking me.
5. **Day 5 morning checkpoint is absolute**: If the backend is not finished, complete it before starting UI and cut a *Should Have* feature instead.
6. **Whoever builds an endpoint writes its authorization test**: Right role passes; wrong role and different hospital fail with 403.
7. **Feature freeze at the end of Day 8**: Days 9–10 are reserved strictly for testing, documentation, and demo rehearsals.
8. **Synthetic data only**: No real patient health information (PHI) in logs, URLs, seed files, or error responses.

---

7. Definition of Done

- [ ] **Concurrency Guarantee**: 20 simultaneous admit requests against 5 beds produce exactly 5 admissions and 15 rejections/waitlist entries; repeating a request with the same `Idempotency-Key` and matching payload hash returns the cached response without creating a second record; reusing an `Idempotency-Key` with a differing request payload hash deterministically returns `422 Unprocessable Entity`; reusing an `Idempotency-Key` across a different scope (different `user_id`, `method`, `route`, or `hospital_id`) does not replay the cached response or suppress the new operation.
- [ ] **Tenant Isolation**: A staff user at Hospital X cannot read, modify, or delete Hospital Y's patients, beds, or queues (enforced by automated test); the city viewer role reads aggregated capacity only.
- [ ] **Audit Completeness**: Every state mutation (admit, transfer, discharge, queue call, referral) writes an immutable `audit_log` row with before/after JSONB state snapshots; sensitive read accesses (patient PHI details view, audit log exports, city referral query) write an immutable `audit_log` row recording the actor, timestamp, accessed resource, hospital scope, action, and UUID request ID with before/after snapshots explicitly set to `NULL`.
- [ ] **Queue Integrity**: Queue tokens are sequential and unique per department per day; call-next respects triage priority with aging so low-priority patients are not starved.
- [ ] **Honest Wait Estimates**: Wait-time estimates are displayed as a range computed from Erlang-C using live doctor count, with an automated fallback when the analytics service is unavailable.
- [ ] **Clean CI Build**: A fresh database builds successfully from migrations in GitHub Actions CI, and one full backup $\to$ restore drill has been executed.
- [ ] **Demo Readiness**: All workflows run smoothly end-to-end in the browser, each demonstrated under a distinct user role.
- [ ] **Academic Documentation**: ER Diagram v2, DFDs (0, 1, 2A–2D), README, threat model, downtime procedure, and final report are complete and aligned with AP course rubrics.